

Запускать так: `./run.sh`

Жадник + TSP solver

Сперва напишем жадник. Возьмём произвольную перестановку вершин. Будем добавлять покупателей какой-то машине в порядке этой перестановки. На момент добавления i -ого покупателя для каждой машины уже есть какой-то маршрут, для всех возможных пар соседних в порядке объезда покупателей какой-то машиной попытаемся вместо того, чтобы ехать от j -ому в порядке объезда к $j + 1$ -ому, между ними заехать к i -ому. Выберем лучшее место, где так можно сделать, на это место и добавим покупателя.

В теории может оказаться так, что из-за нашего неаккуратного распределения покупателей по машинам ни одна машина не готова заехать к покупателю из-за переполнения demand. В этом случае бросим исключение и попробуем ещё раз. В целом при достаточном количестве запусков валидное решение найдётся. Такое в основном случалось на втором и пятом тесте, где после запуска на 5000 раз решение было найдено примерно в каждом четвёртом.

Запустив на 5000 случайных перестановках были получены следующие метрики:

463.026, 1206.403, 1000.002, 2443.396, 3592.360, 5099.912.

Что дало 3 балла на пятом тесте. Итого 0, 0, 0, 0, 3, 0.

Дальше была добавлена дорешка, а именно снова возьмём сортировку в каком-то порядке, извлечём вершину из уже имеющегося решения и попытаемся найти ей новое место улучшив метрику. Так будем делать пока находим хоть какое-то место, улучшающее решение на $\varepsilon = 0.01$. Это позволит найти место получше для стартовых вершин, которые добавлялись без общей картины решения. Это улучшило метрики, хоть и не дало прибавления в баллах. Они составили

458.936, 1167.256, 911.355, 1723.840, 3503.252, 2627.997.

Всё так же получив 0, 0, 0, 0, 3, 0.

На этом моменте в чате объявили, что склад это не $(0, 0)$, а первая точка, что само собой улучшило метрики. У этого же решения настоящие метрики равны:

278.73, 615.66, 622.76, 917.96, 2932.58, 2668.57

Что даёт 19 баллов 5, 5, 3, 3, 3, 0.

На данный момент есть процедура перекидывающая предметы между машинами или внутри одной машины и она в том числе ответственная за правильный порядок объезда одной машиной. Однако для каждой конкретной машины имеется конкретная задача tsp, которую можно решить методами из tsp, так и сделаем.

В tsp наилучшим методом был Лин-Керниган с дополнительными ветвлениями, его и напишем. Это дало небольшое улучшение, метрики составили

278.73, 615.66, 635.61, 925.54, 2924.92, 2571.10

Всё также 19 баллов 5, 5, 3, 3, 3, 0. Причём количество вершин в одной машине не превышает 20, так то вероятно решение tsp будет довольно точным и надо исправлять само разбиение.

Генетика

Глобально не понятно как именно надо делить объекты между машинами, решить tsp после уже не составит трудности. Попробуем найти наилучшее разбиение с помощью генетического алгоритма.

Чтобы избежать симметрий будем рассматривать разбиения на какие-то v множеств всех имеющихся объектов без какого-либо порядка на них. Найдём популяцию. Для этого выберем случайные v магазинов и поместим их в различные машины, далее воспользуемся таким соображением: добавляя магазин к какой-то машине вероятно в имеющимся пути $a \rightarrow b$ заменится на $a \rightarrow c \rightarrow b$, поэтому "плохость" добавления магазина в машину это расстояние до двух самых близких магазинов из машины. То, куда добавление самое хорошее, туда и добавим магазин.

Теперь напишем кроссовер: план следующий, вершины на группы с одним и тем же отцом и матерью, их будем пытаться вставлять вместе. Выберем v групп с попарно различными отцом и матерью и положим их в разные машины. После чего всякую другую группу будем добавлять к отцу либо к матери, если доступны оба варианта, то выбирать случайно, если нет никаких вариантов, то в самом конце все такие элементы попытаемся добавить как и в жадника в ту группу, два наименьших расстояния до которой минимальны. Пока выберем популяцию в 20 штук и на каждом шагу будет создавать 10 детей.

Такое решение не дало продвижения на 1, 2 тестах, на третьем улучшило с 635.61 до 598.74, на остальных получило TL.

Сперва разберёмся с первой половиной тестов, оптимум не был найден. Генетике не хватает вариативности до того, как она сойдётся попытаемся добавить мутаций.

Мутация будет заключаться в том, что с некоторой вероятностью элемент будет перемещаться в другую группу, если в ней есть место. Попытаемся настроить генетику в этом случае.

число скрещиваний (популяция размера 20)	тест 1	тест 2	тест 3	тест 4
1	290.07	711.99	671.44	1321.67
5	285.53	666.12	664.45	1186.32
10	300.92	626.63	651.03	1083.10
15	279.49	607.65	567.11	1070.25
20	278.73	615.44	631.89	1193.82

Наилучшие метрики достиглись при 15 потомках.

Теперь разберёмся с вероятностью мутации и с её силой, которая определяется вероятностью перестановки вершины.

p мутации	p перемещения	тест 1	тест 2	тест 3	тест 4
0.1	0.2	278.98	620.25	571.75	1012.14
0.3	0.2	293.88	650.58	650.58	1090.94
0.5	0.2	294.14	630.01	640.70	1171.88
0.1	0.05	278.99	634.02	624.96	956.19
0.3	0.05	278.73	615.44	648.93	1204.36

Наилучшие метрики при низкой вероятности мутации.

В целом решение несмотря на настройку уступает жаднику, так что сочтём подход неудачным, к множествам лучше добавить структуру. Нынешние метрики равны

278.99, 615.44, 637.78, 1040.78, 2000.29, 2727.44.

Что оставило метрики 5, 5, 3, 3, 3, 0, 19 баллов.

Локальный поиск

На предыдущем подходе TSP-solver не давал серьёзных продвижений, а потому откажемся от него и будем пытаться получить наилучший ответ напрямую. Одну локальную оптимизацию перенесём из прошлого алгоритма: вынуть вершину и вставить её на лучшую доступную позицию. Второе локальное изменение возьмём — поменять местами двух людей. У такого решения метрики составили:

278.73, 620.25, 595.92, 909.38, 2370.27, 2290.81.

Что даёт 22 балла 5, 5, 3, 3, 3, 3.

Пока что мы работаем очень быстро, так что будем добавлять что-нибудь ещё. Добавим 2-opt, разрешающий развернуть произвольный отрезок какого-то из маршрутов целиком. Наибольший профит на данный момент даёт замена двух вершин местами. Эта идея взята из Handbook of Constraint Programming, там же предлагается следующее усиление: возьмём два отрезка и попытаемся переставить местами их. Длины отрезков должны не превосходить K , где K — параметр. Для выбранных начал отрезков переберём все K^2 возможных попарных длин и выберем наилучшее возможное изменение. Если оно улучшает метрику, то его и применим.

K	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
2	278.73	607.65	609.28	884.28	1944.54	2274.47
5	278.73	607.65	609.28	866.93	1565.62	2011.37
7	278.73	607.65	609.28	866.93	1492.26	2000.33
10	278.73	607.65	585.12	871.16	1459.92	2008.54
15	278.73	607.65	585.12	871.16	1506.77	2020.42
30	278.73	607.65	585.12	871.16	1506.77	2020.42

После 10 улучшение пропадает, оставим его. Теперь попробуем решить проблему того, что мы слишком быстро останавливаемся, то есть тем, что у нас заканчиваются шаги. Если шаги закончились, то разрешим принять наихудшее из ухудшающих изменений и попробуем ещё. Решение всё ещё должно остаться валидным и запретим принимать решения, если ухудшение превысит процент от текущего максимума. В этом решении остаётся настроить параметры: количество перезапусков алгоритма после ухудшения и максимальное допустимое ухудшение. Ухудшение привяжем к проценту от текущей метрики, чтобы не зависеть от теста. Сперва на 5 повторениях настроим процент:

процент	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
5	278.73	607.65	556.95	871.16	1451.06	1975.52
15	278.73	607.65	536.58	858.50	1459.07	1966.80
30	278.73	607.65	530.32	822.05	1459.92	1959.76
50	278.73	607.65	570.45	840.30	1459.92	1961.61
70	278.73	607.65	549.66	821.11	1452.50	1951.35

Наилучшие метрики достигнуты при 30%, далее метрики убывают, потому что делаются шаги, которые не приближают к лучшей позиции. Остаётся настроить количество повторений, в целом хуже новое повторение сделать не может, так что главное не получить TL, при этом взяв максимальную метрику.

повторения	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6
0	278.73	607.65	585.12	871.16	1459.92	2008.54
5	278.73	607.65	530.32	822.05	1459.92	1959.76
25	278.73	607.65	524.93	821.92	1442.27	1920.36
100	278.73	607.65	524.93	821.92	1430.47	1909.36
400	278.73	607.65	524.61	821.92	1422.16	1891.18
1000	278.73	607.65	524.61	821.92	1405.07	1891.18 (TL) ¹
4000	278.73	607.65	524.61	821.92	1399.83 (TL)	1891.18 (TL)

На ограничении в 120 секунд метрики составляют:

278.73, 607.65, 524.61, 821.92, 1399.83, 1891.18.

Что даёт все верхние пороги 5, 5, 5, 5, 5, 5, 30 баллов.

Tabu Search

Сейчас для пятого теста потребовалось в разы больше итераций до сходимости чем для остальных, иначе бы 28 баллов можно было бы получить используя максимум 15 секунд вместо двух минут. С другой стороны есть проблема, что какое-то очень хорошее изменение со второй половиной магазинов может очень долго не наступать или не наступить вовсе, если какие-то ухудшения есть в начале. Добавим tabu-search, чтобы ухудшения происходили равномернее по всем машинам и магазинам, а не с машинами первых магазинов.

Tabu list заведём именно для машин, постараемся использовать ухудшение в конкретной машине не слишком часто. Настроим время запрета от табу:

повторения	тест 1	тест 2	тест 3	тест 4	тест 5	тест 6	время на тесте 4(с)
0	278.73	607.65	524.61	821.92	1422.16	1891.18	18.59
1	278.73	607.65	524.61	821.92	1422.16	1891.18	18.55
2	278.73	607.65	527.67	819.56	1394.79	1884.29	18.41
3	278.73	607.65	524.93	821.92	1405.68	1885.14	18.16
4	278.73	607.65	524.61	819.56	1416.53	1885.02	18.07
5	278.73	607.65	527.67	819.56	1410.80	1876.92	18.11
8	278.73	607.65	527.98	829.45	1406.69	1883.56	18.07

Наилучшие метрики достигнуты при блокировке на 2 хода. На всех тестах время упало и решение находится достаточно быстро. Всё также полные метрики. В целом на этом и остановимся.

Итог

Полные метрики с хорошим временем дала адаптация локального поиска по окрестности предложенной в Handbook of Constraint Programming на Табу Поиск для грамотного отсека не выгодных ветвей. Метрики у решения составили:

278.73, 607.65, 527.67, 819.56, 1394.78, 1884.29.

Что дало полные 30 баллов 5, 5, 5, 5, 5, 5. Затраченное при этом время это соответственно 2.01, 2.03, 2.42, 15.51, 18.59, 83.27 секунд.

¹TL имеется в виду как остановлено по времени, которое выставлено на 90 секунд.